

# **Unidad 2. Datos básicos para algoritmos y programas**



# Sistema de numeración decimal

0 1 2 3 4 5 6 7 8 9





# Sistema de numeración decimal

$2356_{(10)}$



# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$



# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$6 \cdot 10^0$$



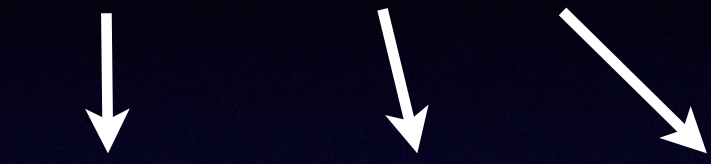
# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$5 \cdot 10^1 + 6 \cdot 10^0$$



# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$



# Sistema de numeración decimal

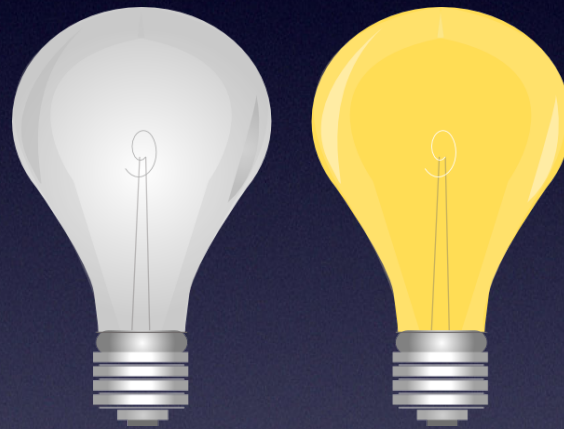
$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$



# Sistema de numeración binario

0 1





# Sistema de numeración binario

0 1





# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$

# Sistema de numeración binario

$$101101_{(2)} =$$

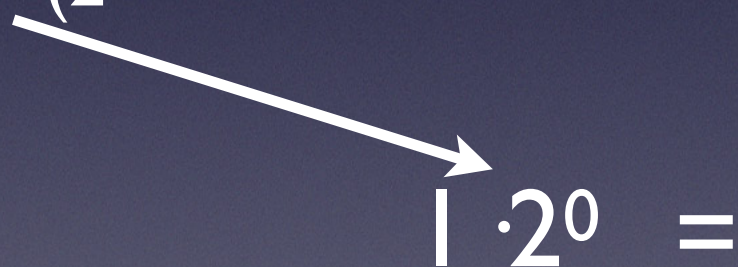


# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$

# Sistema de numeración binario

$$101101_{(2)} =$$

$$1 \cdot 2^0 =$$



# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$

# Sistema de numeración binario

$$101101_{(2)} =$$

$$0 \cdot 2^1 + 1 \cdot 2^0 =$$

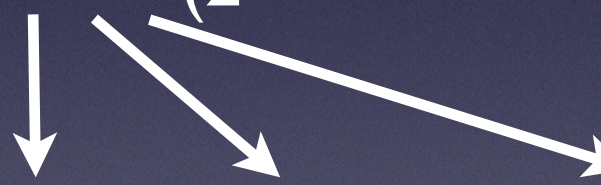


# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$

# Sistema de numeración binario

$$101101_{(2)} =$$

$$1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$$



# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$

# Sistema de numeración binario

$$101101_{(2)} =$$

$$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$$

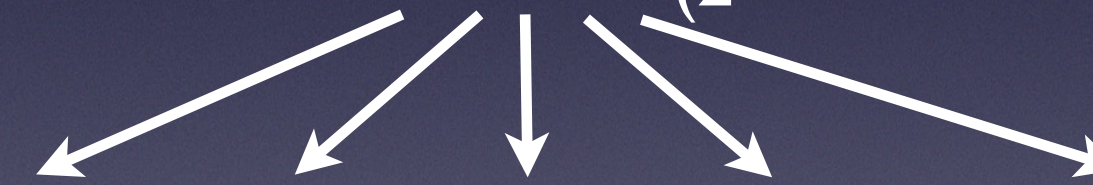


# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$

# Sistema de numeración binario

$$101101_{(2)} =$$

$$0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$$

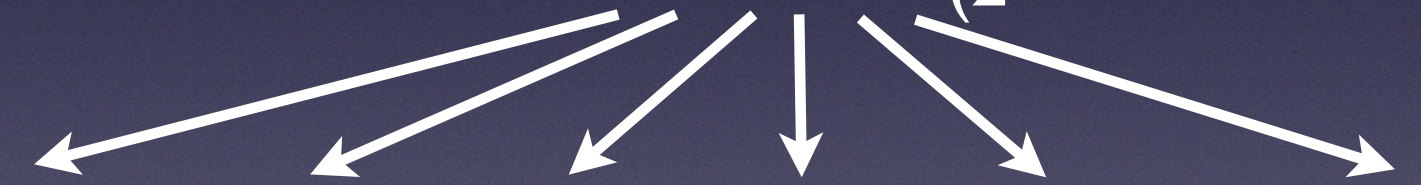


# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$

# Sistema de numeración binario

$$101101_{(2)} =$$

$$1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$$



# Sistema de numeración decimal

$$2356_{(10)} = 2000 + 300 + 50 + 6 =$$

$$2 \cdot 10^3 + 3 \cdot 10^2 + 5 \cdot 10^1 + 6 \cdot 10^0$$

# Sistema de numeración binario

$$101101_{(2)} =$$

$$1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$$

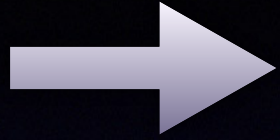
$$32 + 0 + 8 + 4 + 0 + 1 = 45_{(10)}$$



| Decimal | Binary | Explanation |
|---------|--------|-------------|
| 0       | 0      | 0+0+0+0+0   |
| 1       | 1      | 0+0+0+0+1   |
| 2       | 10     | 0+0+0+2+0   |
| 3       | 11     | 0+0+0+2+1   |
| 4       | 100    | 0+0+4+0+0   |
| 5       | 101    | 0+0+4+0+1   |
| 6       | 110    | 0+0+4+2+0   |
| 7       | 111    | 0+0+4+2+1   |
| 8       | 1000   | 0+8+0+0+0   |
| 9       | 1001   | 0+8+0+0+1   |
| 10      | 1010   | 0+8+0+2+0   |
| 11      | 1011   | 0+8+0+2+1   |
| 12      | 1100   | 0+8+4+0+0   |
| 13      | 1101   | 0+8+4+0+1   |
| 14      | 1110   | 0+8+4+2+0   |
| 15      | 1111   | 0+8+4+2+1   |
| 16      | 10000  | 16+0+0+0+0  |
| 17      | 10001  | 16+0+0+0+1  |
| 18      | 10010  | 16+0+0+2+0  |
| 19      | 10011  | 16+0+0+2+1  |
| 20      | 10100  | 16+0+4+0+0  |
| 21      | 10101  | 16+0+4+0+1  |
| 22      | 10110  | 16+0+4+2+0  |

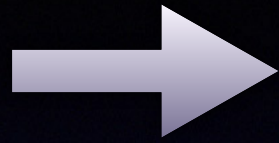


3 bits →

[illegible]



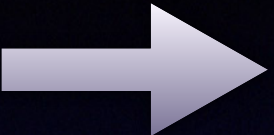
4 bits



|    |   |   |   |   |
|----|---|---|---|---|
| 0  | 0 | 0 | 0 | 0 |
| 1  | 0 | 0 | 0 | 1 |
| 2  | 0 | 0 | 1 | 0 |
| 3  | 0 | 0 | 1 | 1 |
| 4  | 0 | 1 | 0 | 0 |
| 5  | 0 | 1 | 0 | 1 |
| 6  | 0 | 1 | 1 | 0 |
| 7  | 0 | 1 | 1 | 1 |
| 8  | 1 | 0 | 0 | 0 |
| 9  | 1 | 0 | 0 | 1 |
| 10 | 1 | 0 | 1 | 0 |
| 11 | 1 | 0 | 1 | 1 |
| 12 | 1 | 1 | 0 | 0 |
| 13 | 1 | 1 | 0 | 1 |
| 14 | 1 | 1 | 1 | 0 |
| 15 | 1 | 1 | 1 | 1 |

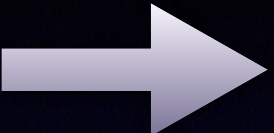


# Mayor número en binario

4 bits   $1111_{(2)}$



# Mayor número en binario

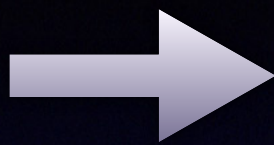
4 bits   $1111_2 =$



$$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$



# Mayor número en binario

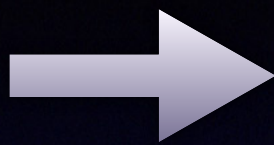
4 bits   $1111_2 = 8+4+2+1_{10}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$





# Mayor número en binario

4 bits   $1111_2 = 8+4+2+1_{(10)} = 15_{(10)}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$





# Mayor número en binario

4 bits  $\longrightarrow$   $1111_2 = 8+4+2+1_{(10)} = 15_{(10)}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

8 bits  $\longrightarrow$   $11111111_2 =$



# Mayor número en binario

4 bits  $\rightarrow$   $1111_2 = 8+4+2+1_{(10)} = 15_{(10)}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

8 bits  $\rightarrow$   $11111111_2 =$

$= 128+64+32+16+8+4+2+1_{(10)}$



# Mayor número en binario

4 bits  $\rightarrow$   $1111_2 = 8+4+2+1_{(10)} = 15_{(10)}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

8 bits  $\rightarrow$   $11111111_2 =$

$= 128+64+32+16+8+4+2+1_{(10)} = 255_{(10)}$



# Mayor número en binario

4 bits  $\rightarrow 1111_2 = 8+4+2+1_{(10)} = 15_{(10)}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

8 bits  $\rightarrow 11111111_2 =$   
 $= 128+64+32+16+8+4+2+1_{(10)} = 255_{(10)}$

3 bits  $\rightarrow 111_2 = 4+2+1_{(10)}$



# Mayor número en binario

4 bits  $\rightarrow 1111_2 = 8+4+2+1_{(10)} = 15_{(10)}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

8 bits  $\rightarrow 11111111_2 =$   
 $= 128+64+32+16+8+4+2+1_{(10)} = 255_{(10)}$

3 bits  $\rightarrow 111_2 = 4+2+1_{(10)} = 7_{(10)}$



# Mayor número en binario

4 bits  $\rightarrow 1111_2 = 8+4+2+1_{(10)} = 15_{(10)}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

8 bits  $\rightarrow 11111111_2 =$   
 $= 128+64+32+16+8+4+2+1_{(10)} = 255_{(10)}$

3 bits  $\rightarrow 111_2 = 4+2+1_{(10)} = 7_{(10)}$

n bits  $\rightarrow$



# Mayor número en binario

4 bits  $\rightarrow 1111_2 = 8+4+2+1_{(10)} = 15_{(10)}$

$1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

8 bits  $\rightarrow 11111111_2 =$   
 $= 128+64+32+16+8+4+2+1_{(10)} = 255_{(10)}$

3 bits  $\rightarrow 111_2 = 4+2+1_{(10)} = 7_{(10)}$

n bits  $\rightarrow 2^n - 1$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 1_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 01_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} +1 \\ 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 001_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} +1 \\ 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 0001_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} +1 \\ 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 10001_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} +1 \\ 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 010001_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} +1 \\ 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 1010001_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

+1

$$\begin{array}{r} 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 11010001_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 111010001_{(2)} \end{array}$$



# Suma de números en binario:

(Igual que en decimal)

$$\begin{array}{r} 11011101_{(2)} \\ + 11110100_{(2)} \\ \hline 111010001_{(2)} \end{array}$$

Ejercicio: Convierte a decimal y comprueba si la suma es correcta



# Números negativos en binario

8 bits



# Números negativos en binario

8 bits

$$00101101_{(2)} = 45_{(10)}$$



# Números negativos en binario

8 bits

$+$    $00101101_{(2)} = 45_{(10)}$



# Números negativos en binario

8 bits

+   $00101101_{(2)} = 45_{(10)}$

-   $10101101_{(2)} = -45_{(10)}$



# Números negativos en binario

8 bits

+   $00101101_{(2)} = 45_{(10)}$

-   $10101101_{(2)} = -45_{(10)}$

Nota: Hay dos representaciones para el 0

00000000

10000000



# Números negativos en binario

8 bits

$$+ \leftarrow \textcircled{0}0101101_{(2)} = 45_{(10)}$$

$$- \leftarrow \textcircled{1}0101101_{(2)} = -45_{(10)}$$

Mayor número  
(Cuando signo)



# Números negativos en binario

8 bits

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

$$- \leftarrow 10101101_{(2)} = -45_{(10)}$$

Mayor número =  $2^7 - 1$   
(Cuando signo)



# Números negativos en binario

8 bits

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

$$- \leftarrow 10101101_{(2)} = -45_{(10)}$$

Mayor número =  $2^7 - 1 = 127$   
(Cuando signo)



# Números negativos en binario

8 bits

$$+ \leftarrow \textcircled{0}0101101_{(2)} = 45_{(10)}$$

$$- \leftarrow \textcircled{1}0101101_{(2)} = -45_{(10)}$$

Mayor número =  $2^{7-1} = 127 \longrightarrow = 2^{n-1-1}$   
(Cuando signo)



# Números negativos en binario

8 bits

$$+ \leftarrow \textcircled{0}0101101_{(2)} = 45_{(10)}$$

$$- \leftarrow \textcircled{1}0101101_{(2)} = -45_{(10)}$$

Mayor número =  $2^{7-1} = 127 \longrightarrow = 2^{n-1-1}$   
(Cuando signo)

Menor número  
(Cuando signo)



# Números negativos en binario

8 bits

$$+ \leftarrow \textcircled{0}0101101_{(2)} = 45_{(10)}$$

$$- \leftarrow \textcircled{1}0101101_{(2)} = -45_{(10)}$$

Mayor número =  $2^7 - 1 = 127 \longrightarrow = 2^{n-1} - 1$   
(Cuando signo)

Menor número =  $-(2^7 - 1)$   
(Cuando signo)



# Números negativos en binario

8 bits

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

$$- \leftarrow 10101101_{(2)} = -45_{(10)}$$

Mayor número =  $2^7 - 1 = 127 \longrightarrow = 2^{n-1} - 1$   
(Cuando signo)

Menor número =  $-(2^7 - 1) = -127$   
(Cuando signo)



# Números negativos en binario

8 bits

$$+ \leftarrow \textcircled{0}0101101_{(2)} = 45_{(10)}$$

$$- \leftarrow \textcircled{1}0101101_{(2)} = -45_{(10)}$$

Mayor número =  $2^7 - 1 = 127 \longrightarrow = 2^{n-1} - 1$   
(Cuando signo)

Menor número =  $-(2^7 - 1) = -127 \longrightarrow = -(2^{n-1} - 1)$   
(Cuando signo)



# Números negativos en binario

8 bits

$$+ \leftarrow \textcircled{0}0101101_{(2)} = 45_{(10)}$$

$$- \leftarrow \textcircled{1}0101101_{(2)} = -45_{(10)}$$

$$\text{Mayor número (Cuando signo)} = 2^7 - 1 = 127 \longrightarrow = \boxed{2^{n-1} - 1}$$

$$\text{Menor número (Cuando signo)} = -(2^7 - 1) = -127 \longrightarrow = \boxed{-(2^{n-1} - 1)}$$



# Números negativos en binario

8 bits

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

$$- \leftarrow 10101101_{(2)} = -45_{(10)}$$

No tenemos suerte.





# Números negativos en binario

8 bits

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

$$- \leftarrow 10101101_{(2)} = -45_{(10)}$$

No tenemos suerte.

Los números negativos  
no se representan así





# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

I. Intercambiar 1s y 0s



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1  $\rightarrow$  11010011<sub>(2)</sub>



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1  $\rightarrow$  11010011<sub>(2)</sub>  
-





# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1  $\rightarrow$  11010011<sub>(2)</sub>

😬: solo hay una representación para el 0  
00000000



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1  $\rightarrow$  11010011<sub>(2)</sub>

😬: solo hay una representación para el 0

1.  $\rightarrow$  00000000  
11111111



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1  $\rightarrow$  11010011<sub>(2)</sub>

😬: solo hay una representación para el 0

00000000

1.  $\rightarrow$  11111111

2.  $\rightarrow$  1 00000000



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1  $\rightarrow$  11010011<sub>(2)</sub>

😬: solo hay una representación para el 0

00000000

1.  $\rightarrow$  11111111

2.  $\rightarrow$  ✗ 00000000



# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1  $\rightarrow$  11010011<sub>(2)</sub>

¿Por qué?





# Números negativos en binario

## Complemento a 2

$$+ \leftarrow 00101101_{(2)} = 45_{(10)}$$

1. Intercambiar 1s y 0s  $\rightarrow$  11010010<sub>(2)</sub>

2. Sumar 1  $\rightarrow$  11010011<sub>(2)</sub>

### ¿Por qué?

Porque la aritmética binaria funciona

Nada especial para sumar y restar números



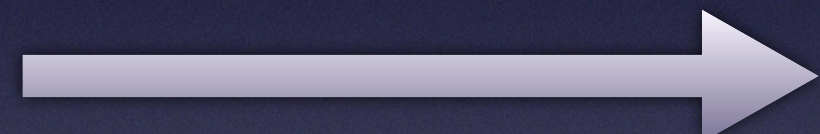


# Números negativos en binario

## Complemento a 2

+   $00101101_{(2)} = 45_{(10)}$

1. Intercambiar 1s y 0s   $11010010_{(2)}$

2. Sumar 1   $11010011_{(2)}$

Mayor número =  $2^7 - 1 = 127$   =  $2^{n-1} - 1$   
(Complemento a 2)



# Números negativos en binario

## Complemento a 2

+  $\leftarrow 00101101_{(2)} = 45_{(10)}$

1. Intercambiar 1s y 0s  $\rightarrow 11010010_{(2)}$

2. Sumar 1  $\rightarrow 11010011_{(2)}$

Mayor número =  $2^7 - 1 = 127 \rightarrow = 2^{n-1} - 1$   
(Complemento a 2)

Menor número =  $-2^7 = -128 \rightarrow = -2^{n-1}$   
(Complemento a 2)



# Números negativos en binario

## Complemento a 2

+  $\leftarrow 00101101_{(2)} = 45_{(10)}$

1. Intercambiar 1s y 0s  $\rightarrow 11010010_{(2)}$

2. Sumar 1  $\rightarrow 11010011_{(2)}$

Mayor número =  $2^7 - 1 = 127 \rightarrow = 2^{n-1} - 1$   
(Complemento a 2)

Menor número =  $-2^7 = -128 \rightarrow = -2^{n-1}$   
(Complemento a 2)  
 $10000000_{(2)} = -128$



# Valores booleanos

- True / False
- 1 / 0
- Yes / No
- On / Off



# Álgebra de Boole

Es un conjunto con solo 2 elementos

1, 0

TRUE, FALSE

Y 3 operaciones

AND, OR, NOT

$\cdot$ ,  $+$ ,  $-$

$\&\&$ ,  $\|\|$ ,  $!$



# Álgebra de Boole

## Leyes fundamentales

OR

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \overline{A} = 1$$

AND

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \overline{A} = 0$$

NOT

$$\overline{0} = 1$$

$$\overline{1} = 0$$



# Tablas de verdad de los operadores booleanos

**F** -> False

**T** -> True

| A | ! A |
|---|-----|
| F | T   |
| T | F   |

NOT

|                |   |   |
|----------------|---|---|
| $\overline{0}$ | = | 1 |
| $\overline{1}$ | = | 0 |



# Tablas de verdad de los operadores booleanos

| A | B | A && B |
|---|---|--------|
| F | F | F      |
| F | T | F      |
| T | F | F      |
| T | T | T      |

AND

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \overline{A} = 0$$



# Tablas de verdad de los operadores booleanos

| A | B | A    B |
|---|---|--------|
| F | F | F      |
| F | T | T      |
| T | F | T      |
| T | T | T      |

OR

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + \underline{A} = A$$

$$A + \overline{A} = 1$$



# Álgebra de Boole

## Leyes fundamentales

OR

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \overline{A} = 1$$

AND

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \overline{A} = 0$$

NOT

$$\overline{0} = 1$$

$$\overline{1} = 0$$



# Álgebra de Boole

## Leyes fundamentales

OR

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \overline{A} = 1$$

AND

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \overline{A} = 0$$

NOT

$$\overline{0} = 1$$

$$\overline{1} = 0$$

Ley conmutativa

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$



# Álgebra de Boole

## Leyes fundamentales

OR

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \overline{A} = 1$$

AND

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \overline{A} = 0$$

NOT

$$\overline{0} = 1$$

$$\overline{1} = 0$$

Ley asociativa

$$(A + B) + C = A + (B + C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$



# Álgebra de Boole

## Leyes fundamentales

OR

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \overline{A} = 1$$

AND

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \overline{A} = 0$$

NOT

$$\overline{0} = 1$$

$$\overline{1} = 0$$

Ley distributiva

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Diferente que en la suma aritmética +



# Álgebra de Boole

## Leyes fundamentales

OR

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \overline{A} = 1$$

AND

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \overline{A} = 0$$

NOT

$$\overline{0} = 1$$

$$\overline{1} = 0$$

Leyes de Morgan

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$



# Álgebra de Boole

No aprender esto:

Propiedades útiles

$$A + (A \cdot B) = A$$

$$A \cdot (A + B) = A$$

$$A + (\overline{A} \cdot B) = A + B$$

$$(A + B) \cdot (A + \overline{B}) = A$$

$$(A + B) \cdot (A + C) = A + (B \cdot C)$$

$$A + B + (A \cdot \overline{B}) = A + B$$

$$(A \cdot B) + (B \cdot C) + (\overline{B} \cdot C) = (A \cdot B) + C$$

$$(A \cdot B) + (A \cdot C) + (\overline{B} \cdot C) = (A \cdot B) + C$$



# Operadores de Igualdad y relacionales en Java

|    |                   |
|----|-------------------|
| == | Igual que         |
| != | No igual que      |
| >  | Mayor que         |
| >= | Mayor o igual que |
| <  | Menor que         |
| <= | Menor o igual que |

Devuelven un  
booleano



# Operadores booleanos

| Operador                       | Símbolo | Acción                              |
|--------------------------------|---------|-------------------------------------|
| Operador lógico<br>Complemento | !       | Invierte el valor de<br>un booleano |
| Y-Condicional<br>AND           | &&      | Ejecuta un Y lógico                 |
| O-Condicional<br>OR            |         | Ejecuta un O<br>lógico              |

Operan con  
booleanos

Y devuelven  
un booleano



# Expresiones booleanas

| Expresión               | Resultado |
|-------------------------|-----------|
| 9 == (3 * 3)            |           |
| 3 != 2                  |           |
| (9 == (3*3)) && (3 !=2) |           |
| (3>2) && ('b'<'a')      |           |
| (3>2)    ('b'<'a')      |           |
| !('a'<'b')              |           |
| (5>1) && !('b'<'a')     |           |



# Expresiones booleanas

| Expresión               | Resultado    |
|-------------------------|--------------|
| 9 == (3 * 3)            | T            |
| 3 != 2                  | T            |
| (9 == (3*3)) && (3 !=2) | T && T -> T  |
| (3>2) && ('b' < 'a')    | T && F -> F  |
| (3>2)    ('b' < 'a')    | T    F -> T  |
| !('a' < 'b')            | !T -> F      |
| (5>1) && !('b' < 'a')   | T && !F -> T |



# Sistema de numeración Hexadecimal

0011 0101 1101<sub>(2)</sub>



# Sistema de numeración Hexadecimal

$\boxed{0011} \boxed{0101} \boxed{1101}_{(2)}$



# Sistema de numeración Hexadecimal

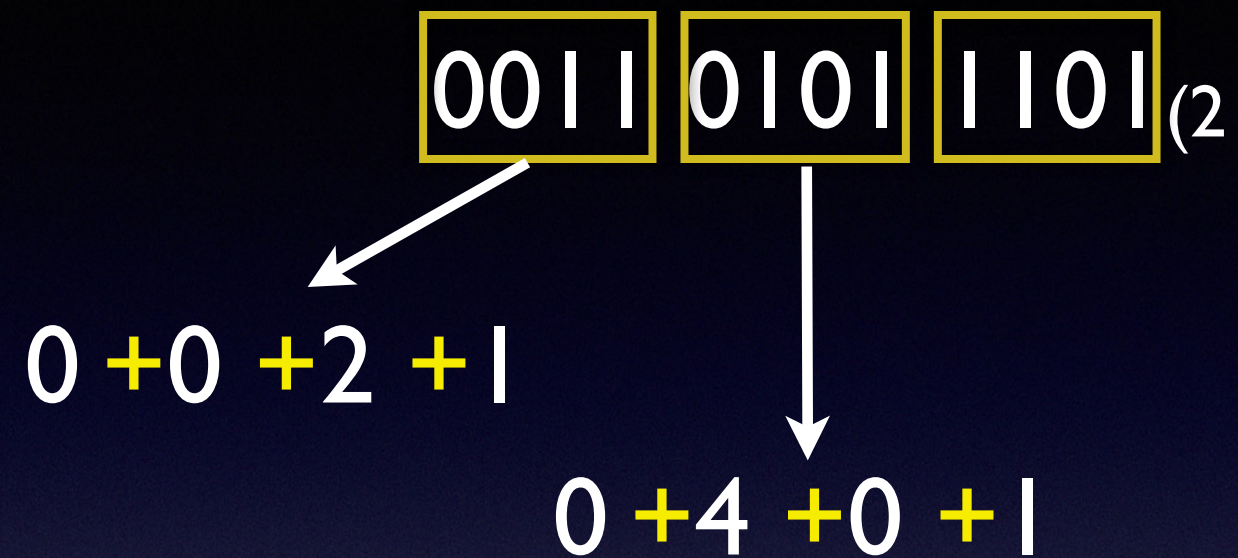
001101011101<sub>(2)</sub>



0 + 0 + 2 + 1

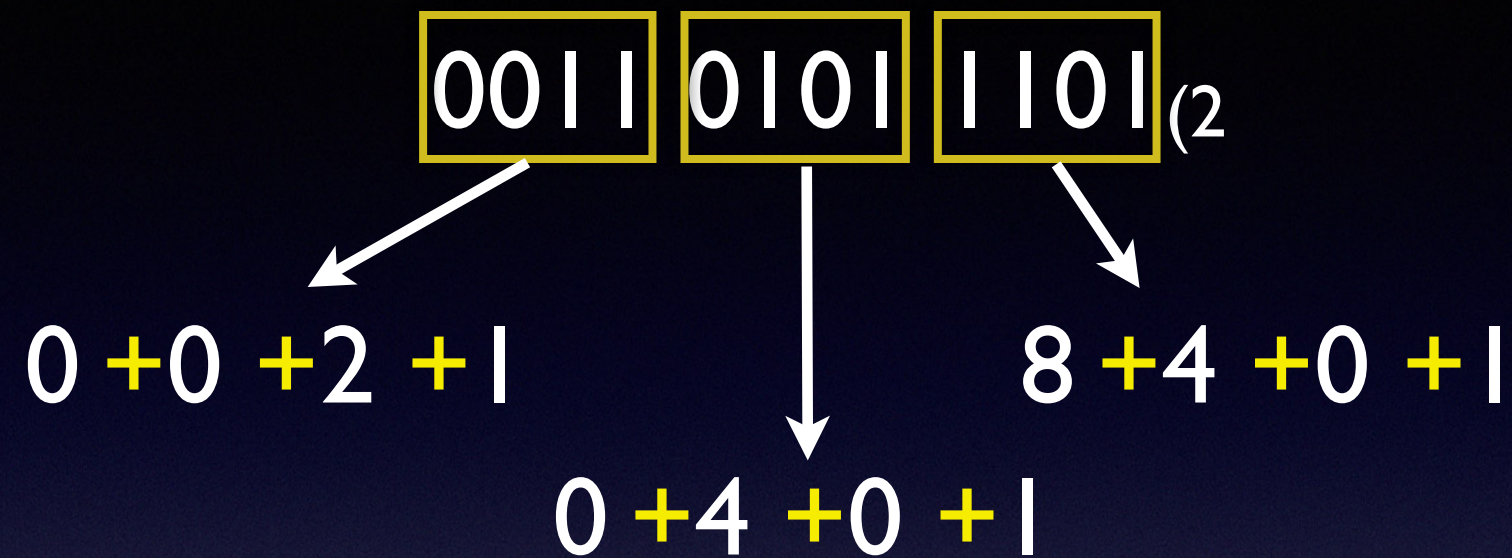


# Sistema de numeración Hexadecimal



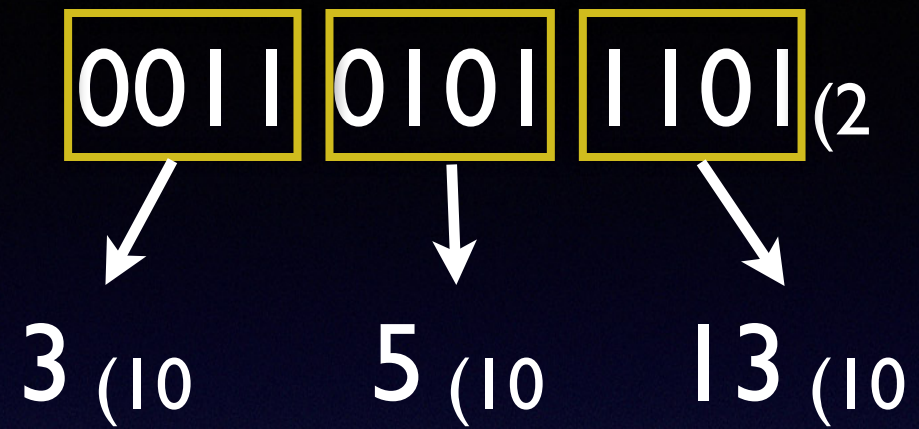


# Sistema de numeración Hexadecimal



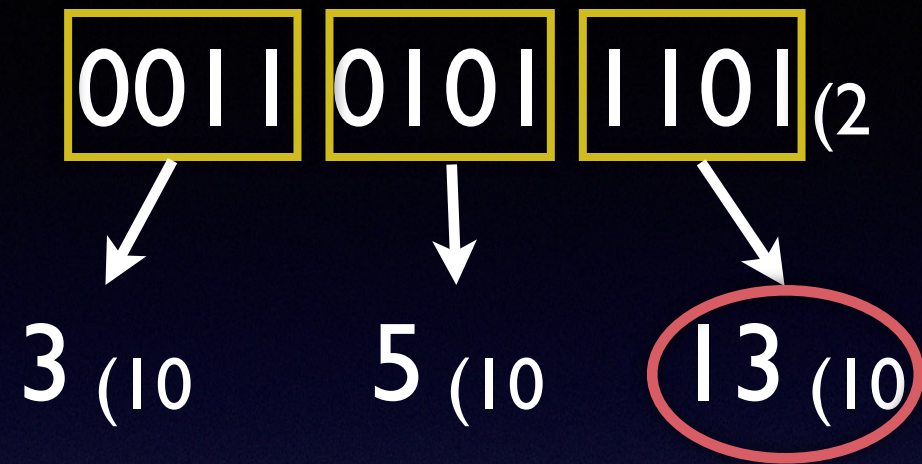


# Sistema de numeración Hexadecimal



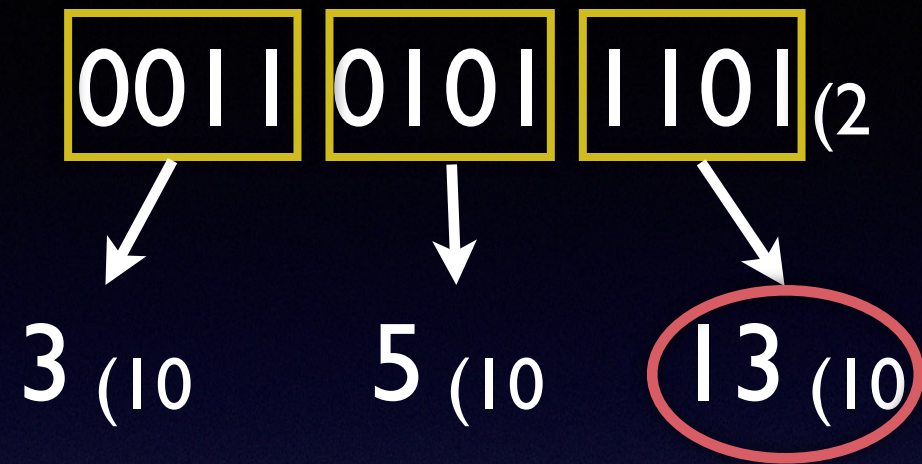


# Sistema de numeración Hexadecimal





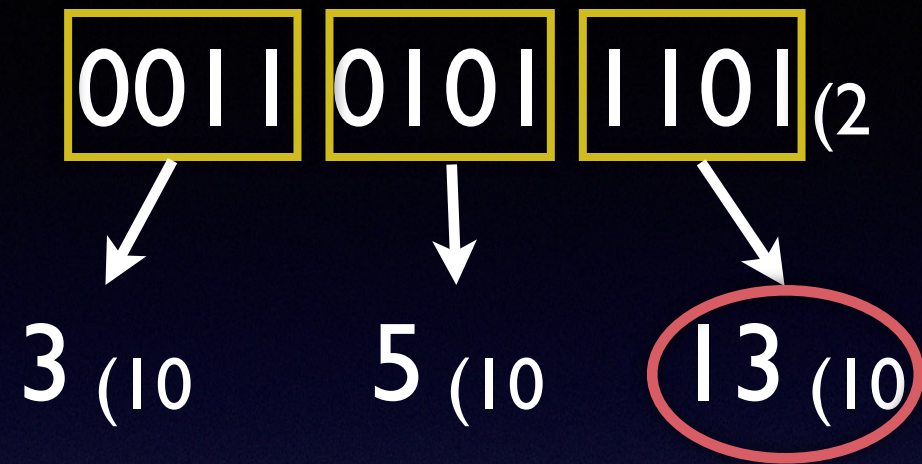
# Sistema de numeración Hexadecimal



Mayor número con 4 bits =



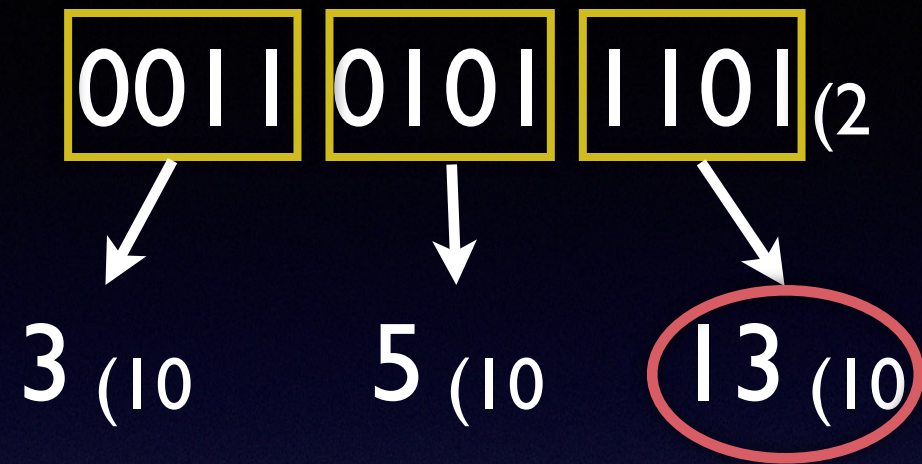
# Sistema de numeración Hexadecimal



Mayor número con 4 bits =  $2^4 - 1 = 15$



# Sistema de numeración Hexadecimal

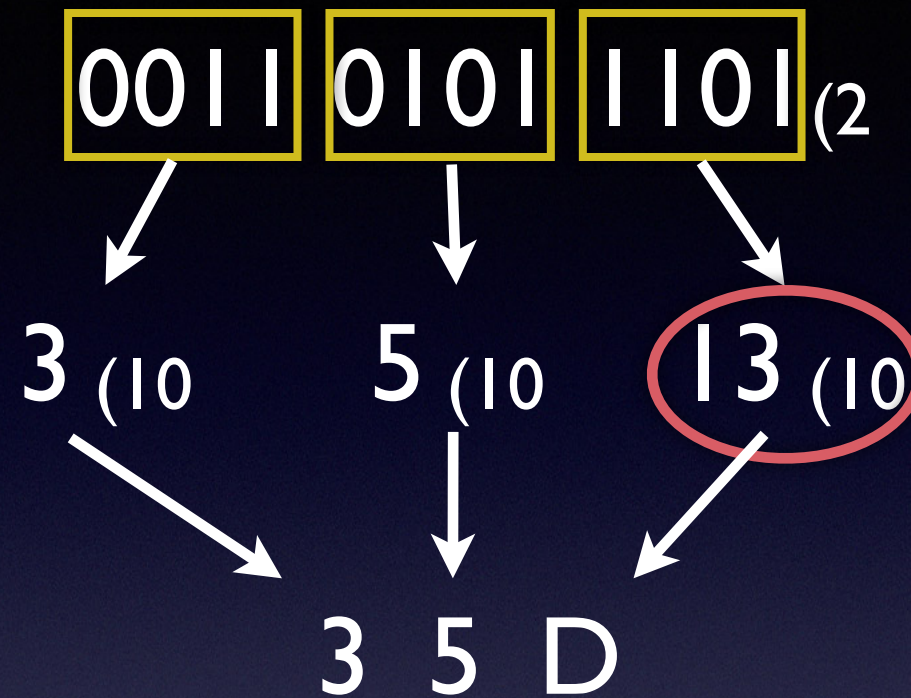


Mayor número con 4 bits =  $2^4 - 1 = 15$

0 1 2 3 4 5 6 7 8 9 A B C D E F



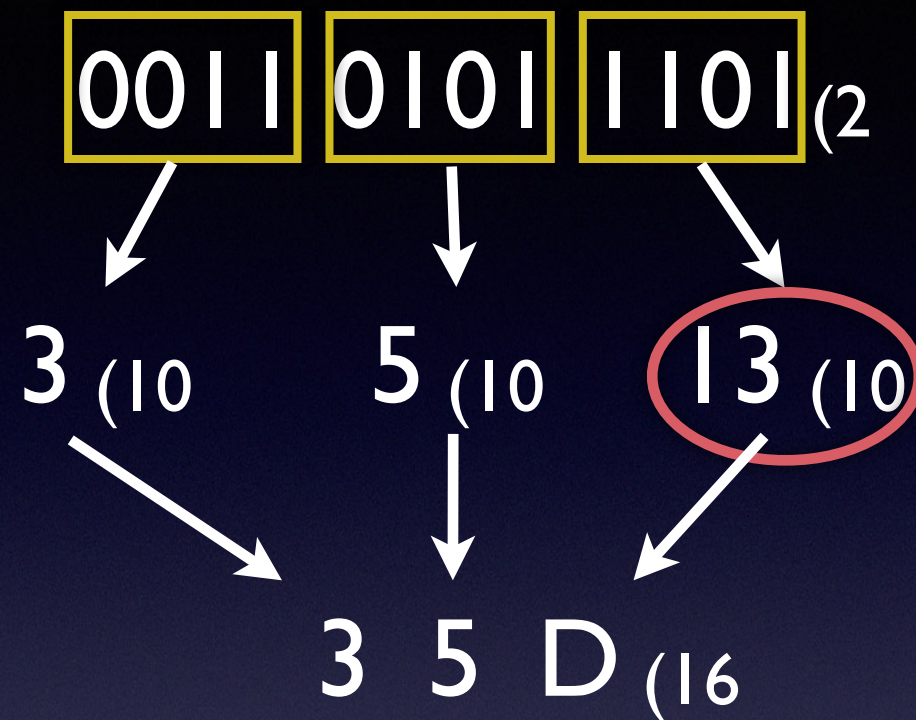
# Sistema de numeración Hexadecimal



0 1 2 3 4 5 6 7 8 9 A B C D E F



# Sistema de numeración Hexadecimal



0 1 2 3 4 5 6 7 8 9 A B C D E F



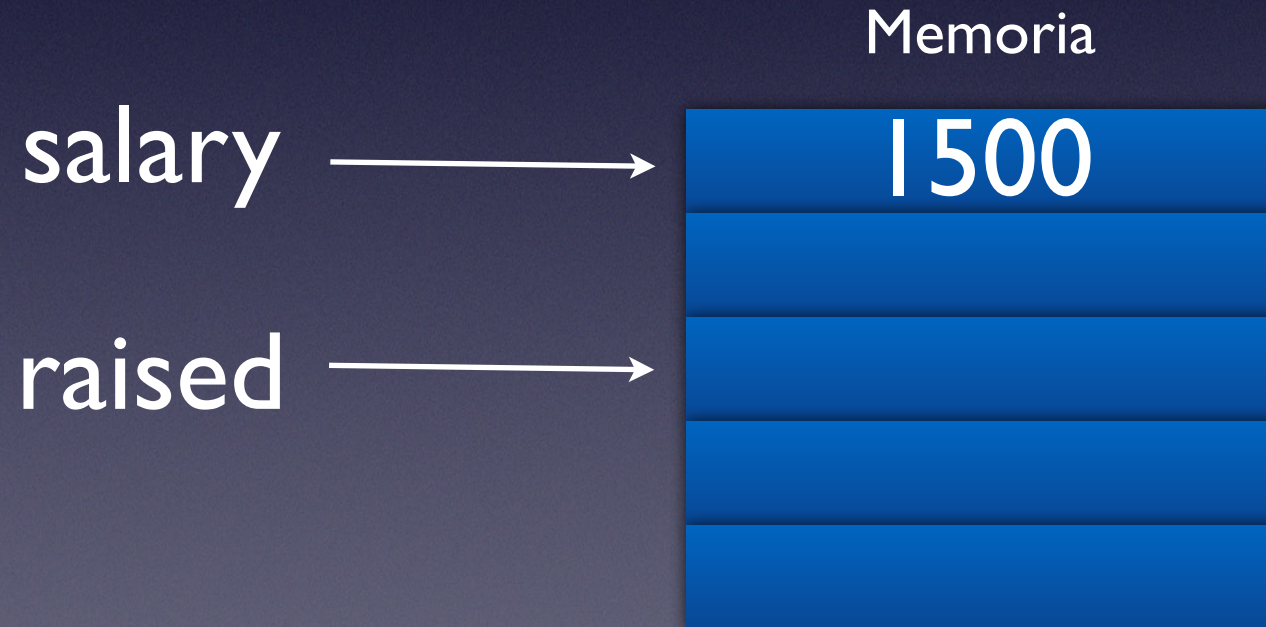
# Variables

Las variables y constantes son ubicaciones de memoria que contienen valores a los que se puede acceder desde un programa usando un nombre o identificador.



# Variables

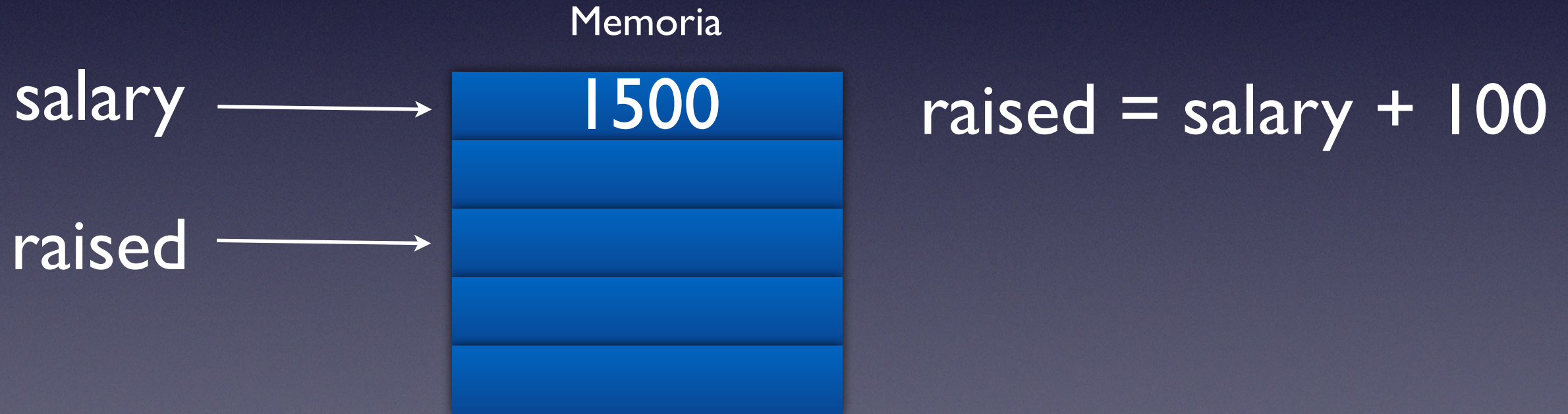
Las variables y constantes son ubicaciones de memoria que contienen valores a los que se puede acceder desde un programa usando un nombre o identificador.





# Variables

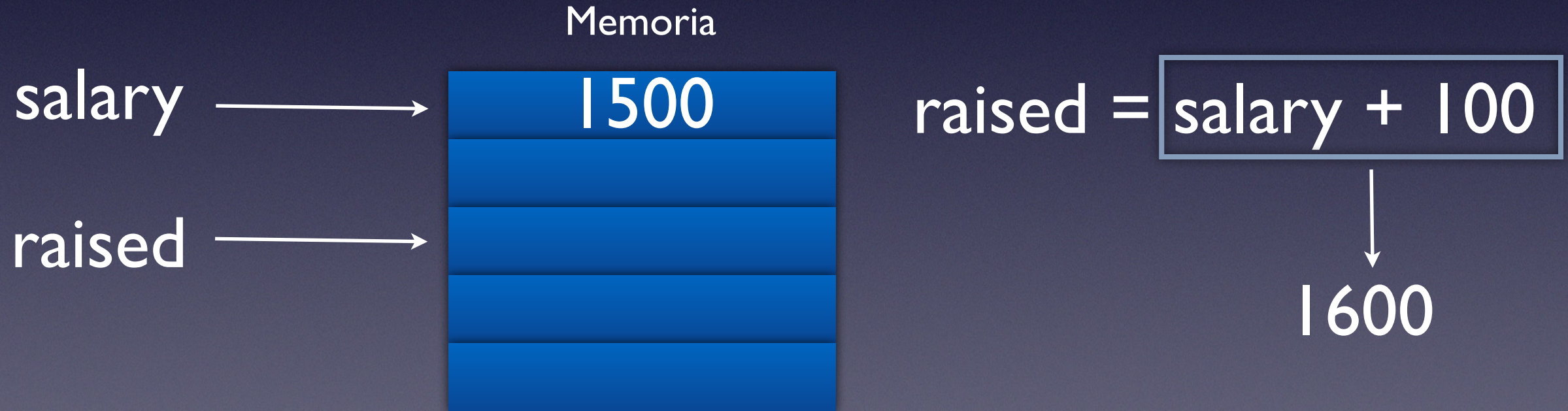
Las variables y constantes son ubicaciones de memoria que contienen valores a los que se puede acceder desde un programa usando un nombre o identificador.





# Variables

Las variables y constantes son ubicaciones de memoria que contienen valores a los que se puede acceder desde un programa usando un nombre o identificador.





# Variables

Las variables y constantes son ubicaciones de memoria que contienen valores a los que se puede acceder desde un programa usando un nombre o identificador.





# Variables

**Las variables y constantes son ubicaciones de memoria que contienen valores a los que se puede acceder desde un programa usando un nombre o identificador.**

**Las variables pueden contener tipos de datos simples (reales, enteros, de carácter, booleanos, etc.) o pueden contener estructuras de datos más complejas (vectores, matrices, registros, etc.).**



# Variables

La **declaración** de las variables se realiza para indicar el nombre de cada variable (también conocido como identificador) y el tipo de datos que almacenarán.



# Variables

Las **variables** se declaran utilizando el tipo de datos y luego el identificador de la variable:

```
<tipo de dato> <identificador>;
```

También se pueden inicializar:

```
<tipo de dato> <identificador> = <value>;
```



# Variables

Las **variables** se declaran utilizando el tipo de datos y luego el identificador de la variable:

`<tipo de dato> <identificador>;`

También se pueden inicializar:

`<tipo de dato> <identificador> = <value>;`

```
int age = 35; // initialized variable
int days;
float temperature;
boolean firstTime;
String name;
```



# Constantes

Las **constantes** son variables que no pueden cambiar su valor. Su valor se asigna en el momento de su declaración.



# Constantes

Las **constantes** son variables que no pueden cambiar su valor. Su valor se asigna en el momento de su declaración

Para declarar una **constante** usamos la palabra reservada **final**:



```
final <data type> <identifier> = <value>;
```



# Constantes

Las **constantes** son variables que no pueden cambiar su valor. Su valor se asigna en el momento de su declaración

Para declarar una **constante** usamos la palabra reservada **final**:

```
final <data type> <identifier> = <value>;
```

```
final float PI=3.14;
```

```
final int DAYS_YEAR = 365;
```

```
final float MAX_TEMPERATURE = 180.55;
```



# Tipos de datos básicos en Java

| Tipo básico | Qué almacena                       | Rango                                                   |
|-------------|------------------------------------|---------------------------------------------------------|
| byte        | enteros de 8 bits                  | -128 to 127                                             |
| short       | enteros de 16 bits                 | -32,768 to 32,767                                       |
| int         | enteros de 32 bits                 | -2,147,483,648 to 2,147,483,647                         |
| long        | enteros de 64 bits                 | -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |
| float       | reales en coma flotante de 32 bits | 6 dígitos significativos ( $10^{-46}$ to $10^{38}$ )    |
| double      | reales en coma flotante de 64 bits | 15 dígitos significativos ( $10^{-324}$ to $10^{308}$ ) |
| char        | carácter Unicode de 16 bits        |                                                         |
| boolean     | variable booleana                  | true / false                                            |

String



# Nombres

*“Pero Elodin el Grande conocía los nombres de todas las cosas, y así todas las cosas estaban a su disposición.”*

*Patrick Rothfuss - El nombre del viento*

***“Los nombres son la forma del mundo, y un hombre que puede pronunciarlos está en camino al poder”***

*Patrick Rothfuss - El nombre del viento*

- “Why do you call this Belows?” I asked Auri.
- “That’s its name,” she said easily. The wind made her fine hair stream out behind her like a gauzy pennant. “You call things by their names. That’s what names are”

*Patrick Rothfuss - The name of the wind*



# Nombres

- **Distingue mayúsculas y minúsculas**



# Nombres

- **Distingue mayúsculas y minúsculas**
- **Primer carácter una letra (Unicode)**



# Nombres

- **Distingue mayúsculas y minúsculas**
- **Primer carácter una letra (Unicode)**
- **El resto de caracteres: cualquier letra, número o \_**



# Nombres

- **Distingue mayúsculas y minúsculas**
- **Primer carácter una letra (Unicode)**
- **El resto de caracteres: cualquier letra, número o \_**
- **Prohibidas las palabras reservadas (if, while, etc..)**



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Variables**:



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Variables**:

- Primer carácter en minúscula



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Variables**:

- Primer carácter en minúscula
- Palabras separadas: primera letra minúscula



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Variables**:

- Primer carácter en minúscula
- Palabras separadas: primera letra minúscula

```
variableNumberOne  
backgroundColor  
fixedSalaryComponent
```



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Constantes**:



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Constantes**:

- Todo en mayúsculas



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Constantes**:

- Todo en mayúsculas
- Palabras separadas: unidas por subrayado \_



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Constantes**:

- Todo en mayúsculas
- Palabras separadas: unidas por subrayado \_

PI



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Constantes**:

- Todo en mayúsculas
- Palabras separadas: unidas por subrayado \_

```
PI  
NUMBER_OF_DAYS
```



# Nombres

- Distingue mayúsculas y minúsculas
- Primer carácter una letra (Unicode)
- El resto de caracteres: cualquier letra, número o \_
- Prohibidas las palabras reservadas (if, while, etc..)

Convenciones para las **Constantes**:

- Todo en mayúsculas
- Palabras separadas: unidas por subrayado \_

```
PI  
NUMBER_OF_DAYS  
MAXIMUM_VALUE
```



# Literales

Un *literal* es la representación en código fuente de un valor fijo; los literales se representan directamente en el código sin necesidad de cálculos. Como se muestra a continuación, es posible asignar un literal a una variable de tipo primitivo:

```
boolean result = true;
char capitalC = 'C';
byte b = 103;
short s = 10240;
int i = 105070;
// The number 26, in decimal
int decVal = 26;
// The number 26, in hexadecimal
int hexVal = 0x1a;
double d1 = 123.4;
// same value as d1, but in scientific notation
double d2 = 1.234e2;
float f1 = 123.4f;
```



# Operadores

= Operador de **asignación** simple  
**Es el operador más importante.**

**Se usa para asignar un valor o una expresión a una variable o constante.**



# Operadores

## Operadores **Aritméticos**

- + Suma (usado también para la concatenación de strings)
- Resta
- \* Multiplicación
- / División
- % Resto de la división entera



# Operadores

## Operadores **Unarios**

- + Operador unario positivo; indica valor positivo (opcional)
- Operador unario negativo; niega una expresión aritmetica
- ++ Incremento; incrementa el valor en 1
- Decremento; decrementa el valor en 1
- ! Complemento lógico; Invierte el valor de un booleano



# Operadores

## Operadores **relacionales**

|    |                   |
|----|-------------------|
| == | Igual a           |
| != | No igual a        |
| >  | Mayor que         |
| >= | Mayor o igual que |
| <  | Menor que         |
| <= | Menor o igual que |

Devuelven un booleano



# Operadores

Operadores **Condicionales** (o **booleanos**)

&& Y-Condicional (AND)

|| O-Condicional (OR)

Los operandos son booleanos y devuelven un valor booleano



# Operadores

## Operadores a nivel de **Bit**

- ~ Unario. Complemento de bits
- << Desplazamiento a la izquierda con signo
- >> Desplazamiento a la derecha con signo
- >>> Desplazamiento a la derecha sin signo
- & AND bit a bit
- ^ OR exclusivo bit a bit
- | OR inclusivo bit a bit

Operan a nivel de bits



| Operadores             | Precedencia                            |
|------------------------|----------------------------------------|
| Postfijo               | expr++ expr--                          |
| Unario                 | ++expr --expr +expr -expr ~ !          |
| Multiplicativo         | * / %                                  |
| Aditivo                | + -                                    |
| Desplazamiento         | << >> >>>                              |
| Relacional             | < > <= >= instanceof                   |
| Igualdad               | == !=                                  |
| AND a nivel de bit     | &                                      |
| OR a nivel de bit      | ^                                      |
| OR inclusivo a niv.bit |                                        |
| Y lógico (AND)         | &&                                     |
| O lógico (OR)          |                                        |
| Ternario               | ? :                                    |
| Asignación             | = += -= *= /= %= &= ^=  = <<= >>= >>>= |



Una **expresión** es una construcción compuesta de variables, literales, operadores e invocaciones de funciones, que se construyen de acuerdo con la sintaxis del lenguaje y que evalúan a un único valor.

1. Un **literal** es una expresión. Ejemplo: “Antonio”, 1.23, ...
2. Una **constante** o una variable es una expresión: Ejemplo: PI, x, name, ...
3. Una **función** es una expresión: Ejemplo: sin(x), cos(x), ...
4. Una **combinación** de literales, constantes, variables, funciones y operadores siguiendo las reglas de la sintaxis del lenguaje de programación. Ejemplo:  $\cos(\pi * x) + 1.25 > \sin(y)$



```
int numElements = 0;  
String name = "Antonio";  
System.out.println("Good morning " + name);
```

```
int result = 1 + 2;  
if (value1 == value2)  
    System.out.println("value1 == value2");
```